RAG Programming Assignment

Edwin Buck

2026 Apr 07

Overview

Design, Implment and Document a Resource Augmented Generator (RAG) solution.

The solution should not be production-ready, but should be illustrative of the Key Components and design choices, with clear explanations

Choices are intentionally simplistic

- Complex design is not required to illustrate component understading
- Complex architecture can confound core concepts
- Simple designs have minimal resource requirements
- Know problem domain reduces risk of failure due to domain issues
- Example must run on a standlaone laptop

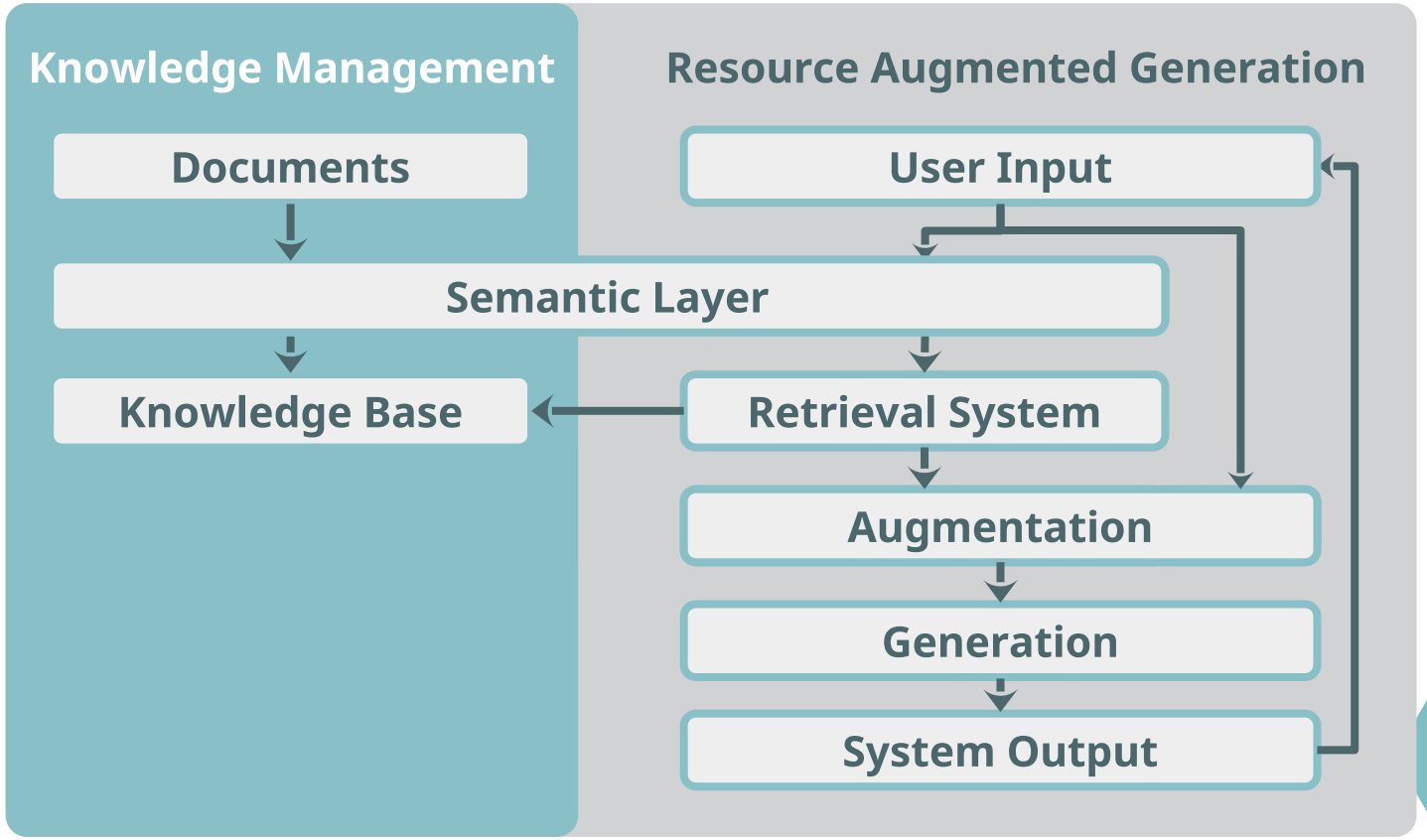
Simplicity has drawbacks

- Lack of flexibilty, unused flexibility is often missing
- Narrow demonstration of skills (and possibly understanding)
- Potentially undervalued solution – the curse of simlpicity

I attempt to avoid the drawbacks by mentioning opportunities to expand the solution, where appropriate

Component Architecture

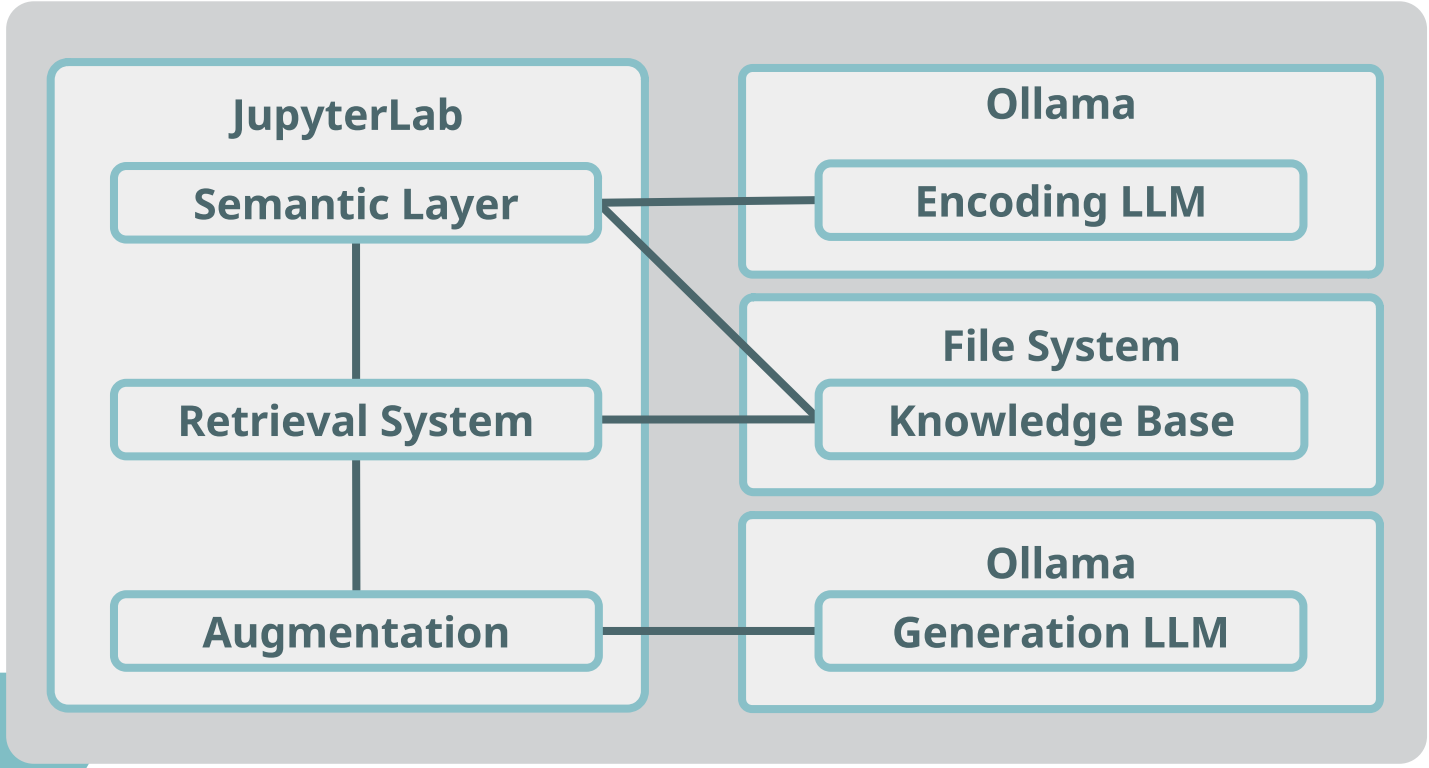
Knowledge Base	Queryable information relevant to user input
Semantic Layer	Translates text for meaningful comparison
Retrieval System	Fetches the most relevant knowledge
Augmentation	Combines relevant information with a query
Generation	Creates response based on augumented input



Deployment Architecture

Deployment is into a single, standalone laptop

Jupyter Lab	Contains all programming and logic flows
Ollama	Contains and hosts all models
File System	Contains a file based knowledge database

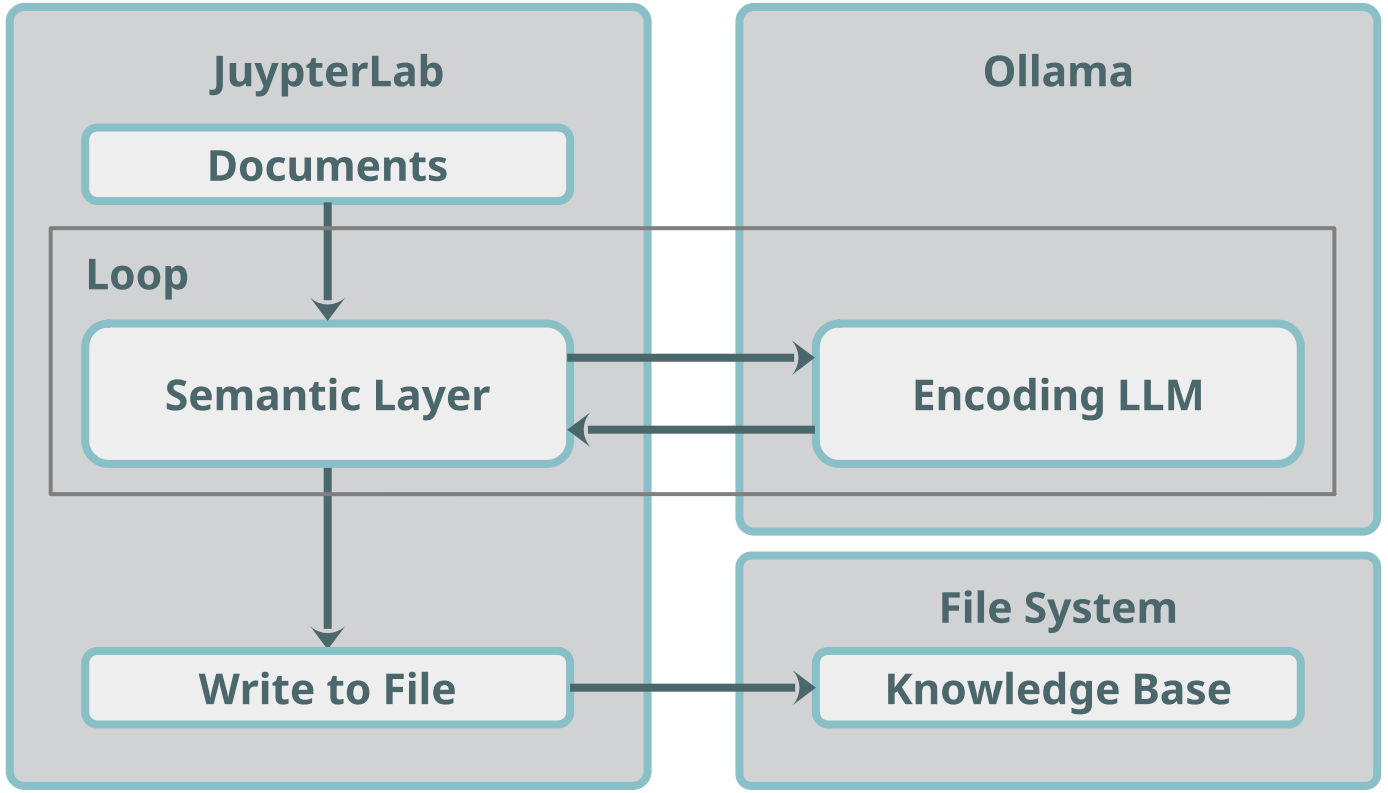


Knowledge Base Construction

Before using a knowledge base, it must be constructed,

Facts are translated into encodings, a vector of numbers aligning with semantic meaning.

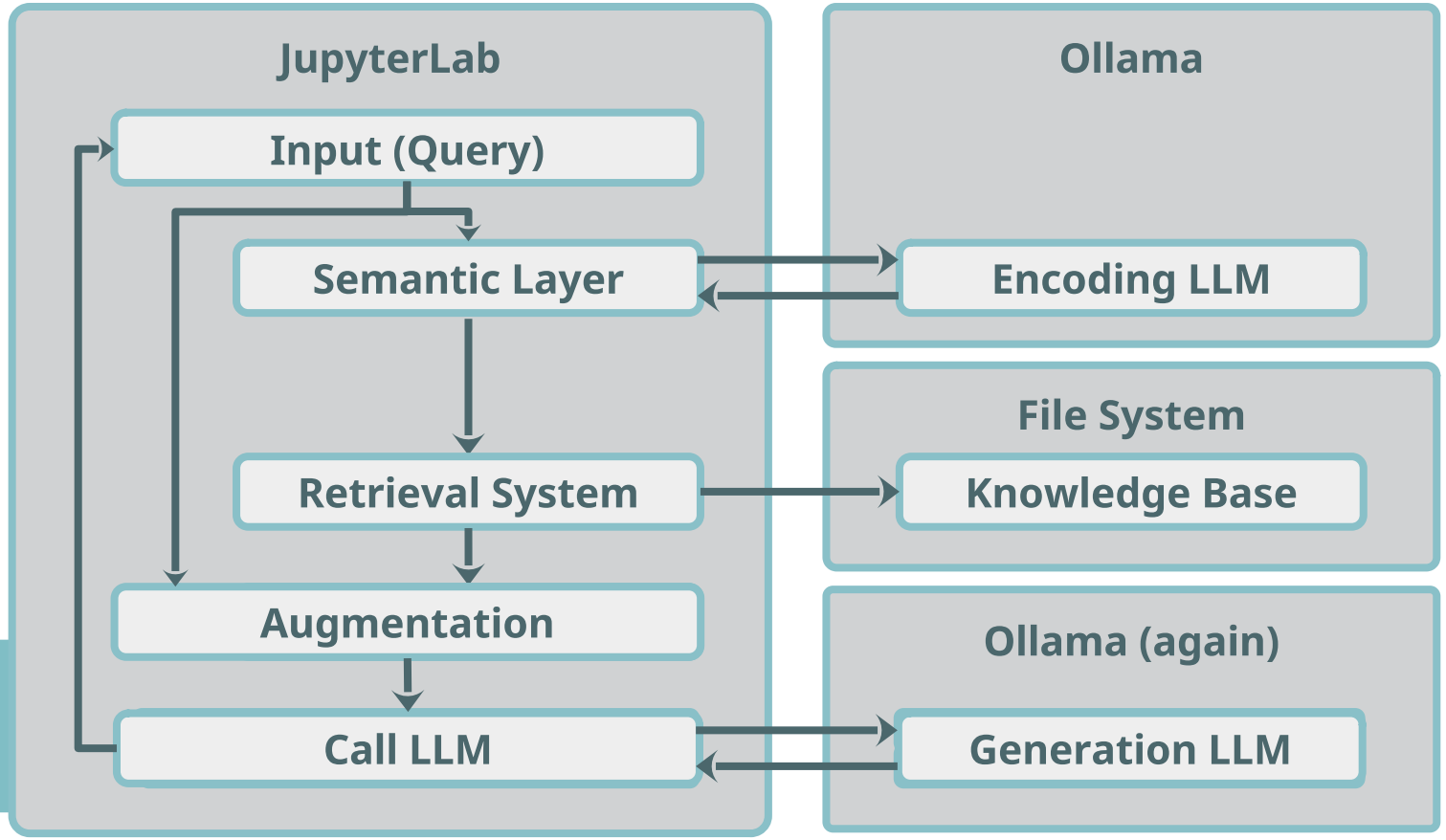
Interactions to Construct the Knowledge Base



Resource Augmented Generation

After the knowledge base is constructed, it can be used.
Queries are translated to semantic encodings.
Semantic encodings are used to search a knowledge base.
Semantically similar items are collected and added to the LLM prompt.
The LLM, augmented with data, should provide improved responses.

Interactions to Leverage the Knowledge Base



Implementation Plan

The plan was, like all good plans, mostly followed.

1) Install JupyterLab
2) Install Ollama
3) Create initial notebook calling Generation LLM
4) Add fact dataset
5) Expand notebook to read facts, encode, and store in array
6) Expand notebook to search array
7) Expand notebook to augment query to Generation LLM
8) Add code to launch LLMs from within Jupyter
9) Add code to store Knowledge Base to file
10) Split notebook into:
    a) Create Knowledge Base notebook
    b) Use RAG notebook.
11) (time permitting) non-interactive testing
12) (time permitting) revalidation of documentation
13) (time permitting) improvement of installation / configuration

In preparing for battle I have always found that plans are useless, but planning is indispensable

- Dwight D. Eisenhower

Challenges

A number of small challenges were encountered.

1) I wrote design documentation too early, just to change course.
2) I believed LLMs might accept encodings and text prompts. This led me to iniitally not retrieve the fact with the encoding.
3) Optimizing the retrieval of the top N vector matches was fun but unnecessary.
4) Shopped for stand alone Vector Databases for too long, even deploying pgvector. Then scrapped it as superfluous.
5) Challenges in adapting production python development patterns to Jupyter Notebooks. (creating modules / unit testing)
6) Unexpected last-minute prompt engineering, to make the responses more natural.
7) Time explorating of tangentially related material.
8) I'm less familiar with LibreOffice Presenter than PowerPoint

Bad times have a scientific value.
These are occasions a good learner would not miss.

- Ralph Waldo Emerson

Questions and Answers

Edwin Buck

2026 Apr 07